

marp: true

theme: default

paginate: true

style: |

/* — Base — */

@import url('https://fonts.googleapis.com/css2?

family=Inter:wght@400;600;700;900&family=JetBrains+Mono:wght@400;700&display=swap');

section {

font-family: 'Inter', 'Segoe UI', sans-serif;

font-size: 20px;

background: #0d1117;

color: #e6edf3;

padding: 48px 64px;

}

The setup

The hardware

 2 vCPU · 2 GB RAM · 20 GB SSD

 ~\$12/month (Hetzner CX22)

 Docker Compose (same constraints, same demo)

The WordPress site

 2,500 posts · 500 WooCommerce products

 ~6,000 revisions in DB

 Realistic bloat: transients, orphaned meta

 WooCommerce + Yoast SEO active

The load generator



The journey

Level	Stack	p95 Latency	requests/s	CPU	DB Threads	New layer
0	Apache + mod_php	💣 crash @ ~200 VU	~40+ → collapse	100%	20+	—
1	+ Nginx · FPM · OPcache · Redis	~100–130 ms	~50–60	~25%	~15–20	OPcache · Redis
2	+ FastCGI cache · MariaDB tuning	~70–90 ms	~65	~18%	~0–3	FastCGI page cache

Key concept: what is a Virtual User?

1 VU = 1 browser-like loop, running forever

```
VU #1:  GET /  → wait 1s → GET /?p=1 → wait 1s → repeat ...
VU #2:  GET /  → wait 1s → GET /?p=2 → wait 1s → repeat ...
      ⋮
VU #50: GET /  → wait 1s → GET /?p=5 → wait 1s → repeat ...
```

50 VUs = 50 open connections = ~50 req/s sustained

Real-world equivalent (*think time = 15–30× longer in real life*)

k6 VUs	≈ Real simultaneous visitors
10	~150 – 300
50	~750 – 1,500
100	~1,500 – 3,000

Key metric definitions

Term	What it actually means
Virtual Users	Concurrent browser-like loops running continuously — not unique visitors
requests/s	Total HTTP responses served per second across all VUs
p95 latency	95% of requests were <i>faster</i> than this — 5% were slower
p99 latency	The near-worst case — 1% of requests were slower than this
FPM workers	PHP processes <i>currently executing code</i> right now
FPM queue depth	Requests waiting for a free PHP worker — 0 = healthy, >0 = trouble
DB threads running	MySQL queries <i>actively executing</i> — not just connected

The two load test scripts

Why does one screenshot say **50 VU** and another say **500 VU**?

snapshot.js — 50 Virtual Users

Purpose: measure and compare levels

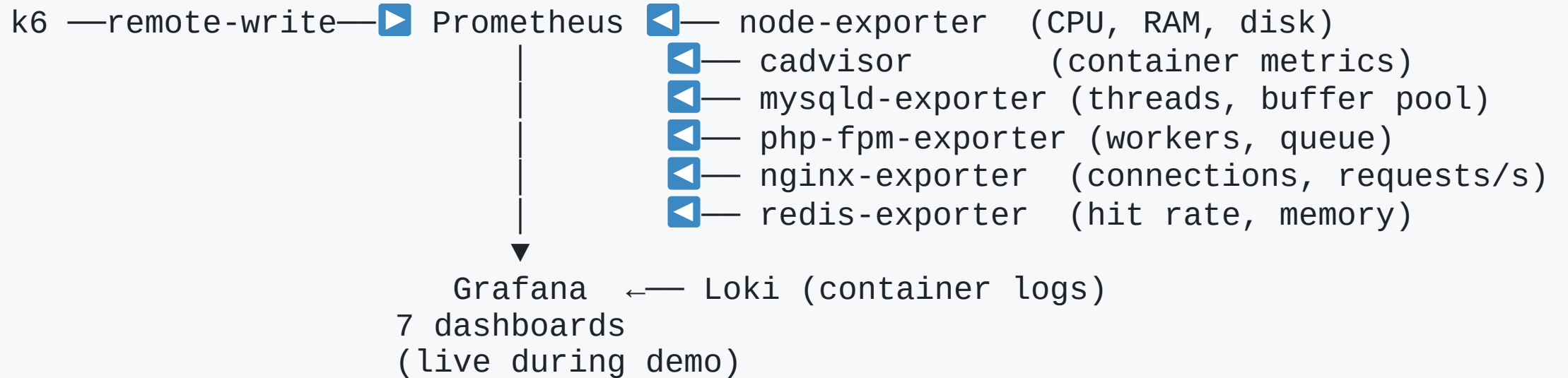
```
0 → 50 VU in 20s  
hold 50 VU for 50s    ← screenshot fires here  
ramp down in 20s  
Total: 90 seconds
```

Used for: `make snapshot-l1`, `make snapshot-l2`

- ✓ All three levels visible and comparable
- ✓ L0 struggles, L1 stable, L2 stays clean

black-friday.js — 0 → 500 Virtual Users

Observability stack



Every number you see on screen is a **real metric from a real process**.

No synthetic benchmarks. No cherry-picked results.



ACT I

The Crash

Level 0 — Apache + mod_php

Level 0 — How Apache prefork works

Each connection = one OS process

Browser connects

- Apache forks a worker process
- worker loads PHP + WordPress into RAM
- PHP runs, queries MySQL
- response sent
- worker stays alive, waiting (idle, ~32 MB RAM held)

The problem:

- 60 idle workers = 1.9 GB RAM consumed
- Browser #61 gets: queue
- Browser #200 gets: **503**

The process model was designed for static files in 1996.

The Apache math

The Apache Math — Why 47 Workers Max

Max safe workers = (Total RAM - Overhead) ÷ RAM per worker
(2,048 MB - 512 MB) ÷ 32 MB/worker = 47 workers max

RAM BREAKDOWN

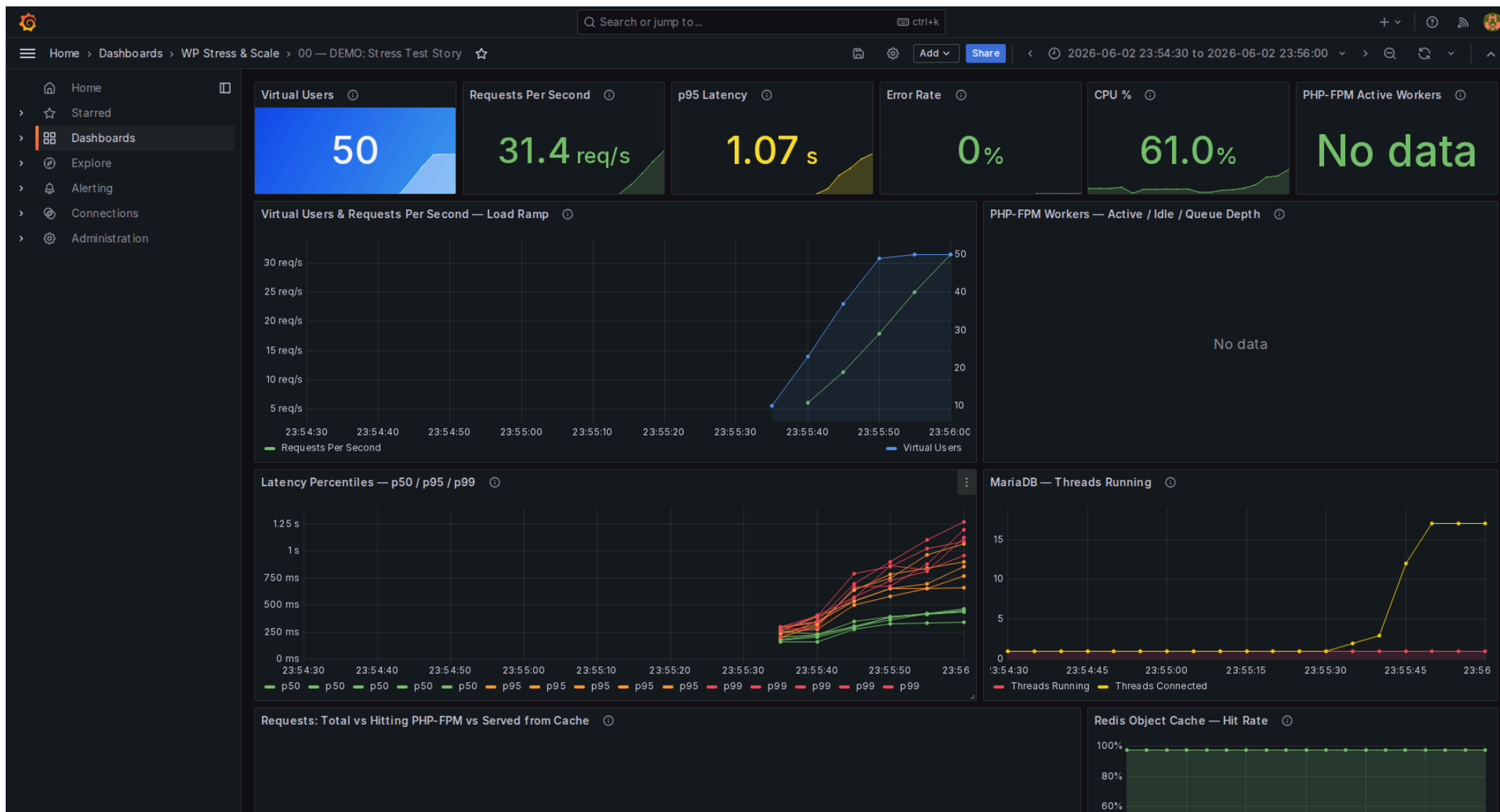
OS + DB + Redis 512 MB (25%)	Workers 1–40 × 32 MB = 1,280 MB	41–47 (224 MB)
---------------------------------	---------------------------------	-------------------

WHAT HAPPENS AS WORKERS FILL UP

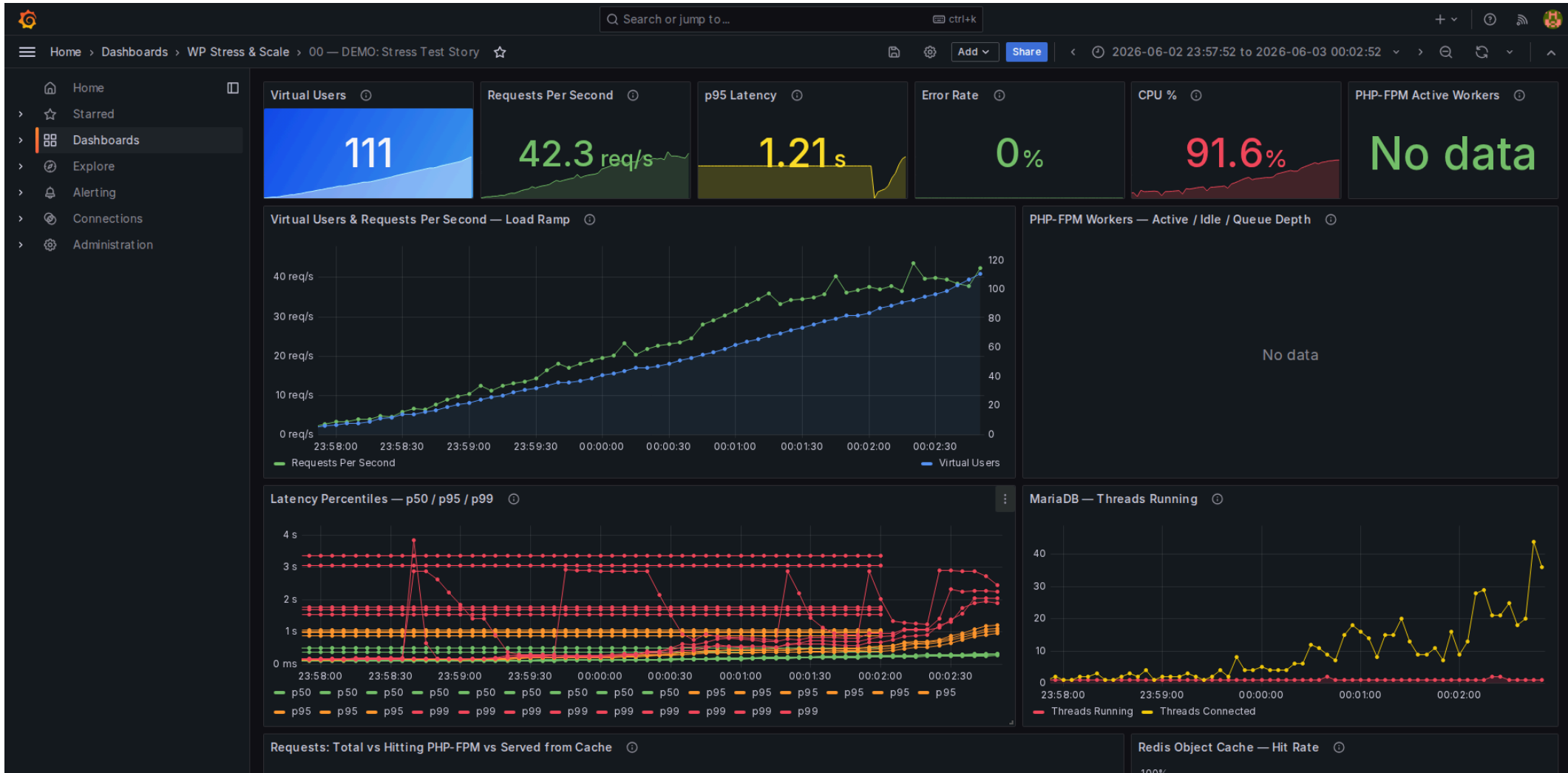
Workers active	What happens
1 – 40	Fast responses — queue depth 0, all requests served immediately
41 – 47	Slow — system starts swapping, response times rise
48	Queue forms — new requests wait for a free worker
100+	503 Service Unavailable — connections refused at the OS level

"This is not a bug. It is arithmetic. The crash is predictable - calculable - from the worker count alone."

Level 0 — Baseline (50 Virtual Users — before the storm)



🔥 Level 0 — The Crash (Black Friday ramp: 0 → 500 VU — Apache dies at ~50)



Level 0 — reading the crash

Panel	Metric	Value	What it means
VU & req/s	req/s at 107 VU (screenshot)	~40 req/s	Still serving — collapse happens past ~200 VU
VU & req/s	req/s trend	~40+ → collapse	Screenshot captured mid-ramp; peak-then-drop happens off-screen
Latency	p95	~3–6 s	Many × worse than Level 1
Latency	p99	~6 s (hard ceiling)	Connections timing out at OS level
Latency	p50	~1–2 s	Even the median is unusable
Apache	Connection backlog	~250+	OS listen queue overflowing — new connections refused



ACT II

Stop the Bleeding

Level 1 — Nginx + PHP-FPM + OPcache + Redis

Level 1 — Three changes, zero cost

1. Nginx (event-driven)

One thread, 10,000+ concurrent connections.

Zero RAM per idle connection.

2. PHP-FPM (decoupled pool)

Workers handle PHP execution only.

Nginx queues connections externally.

`pm.max_children = 20` — hard limit, **never OOM**.

3. OPcache + Redis object cache

- OPcache: compiled PHP bytecode in shared memory — zero disk reads
- Redis: DB query results in RAM — 82% of DB calls served from memory

OPcache — the biggest free win

✗ Without OPcache (Level 0)

Every single request:

```
disk read → parse → lex  
→ AST → compile → opcodes  
→ execute → response
```

Repeat for 300+ PHP files.
wp-settings.php alone
includes 100+ files.

✓ With OPcache (Level 1+)

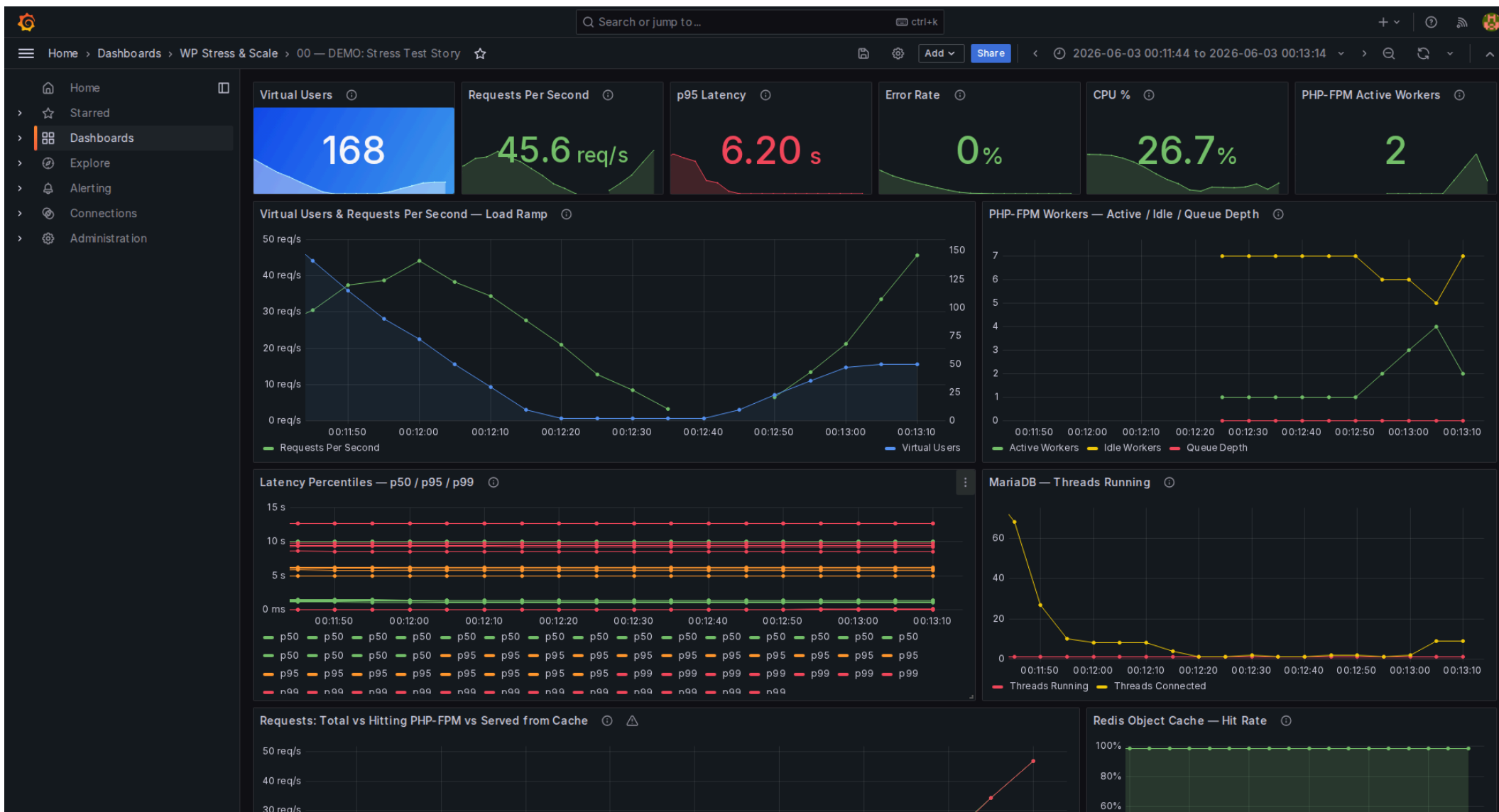
First request only:

```
compile → store in shared RAM
```

Every request after:

```
read opcodes from RAM
```

Level 1 — Demo Dashboard (50 Virtual Users)



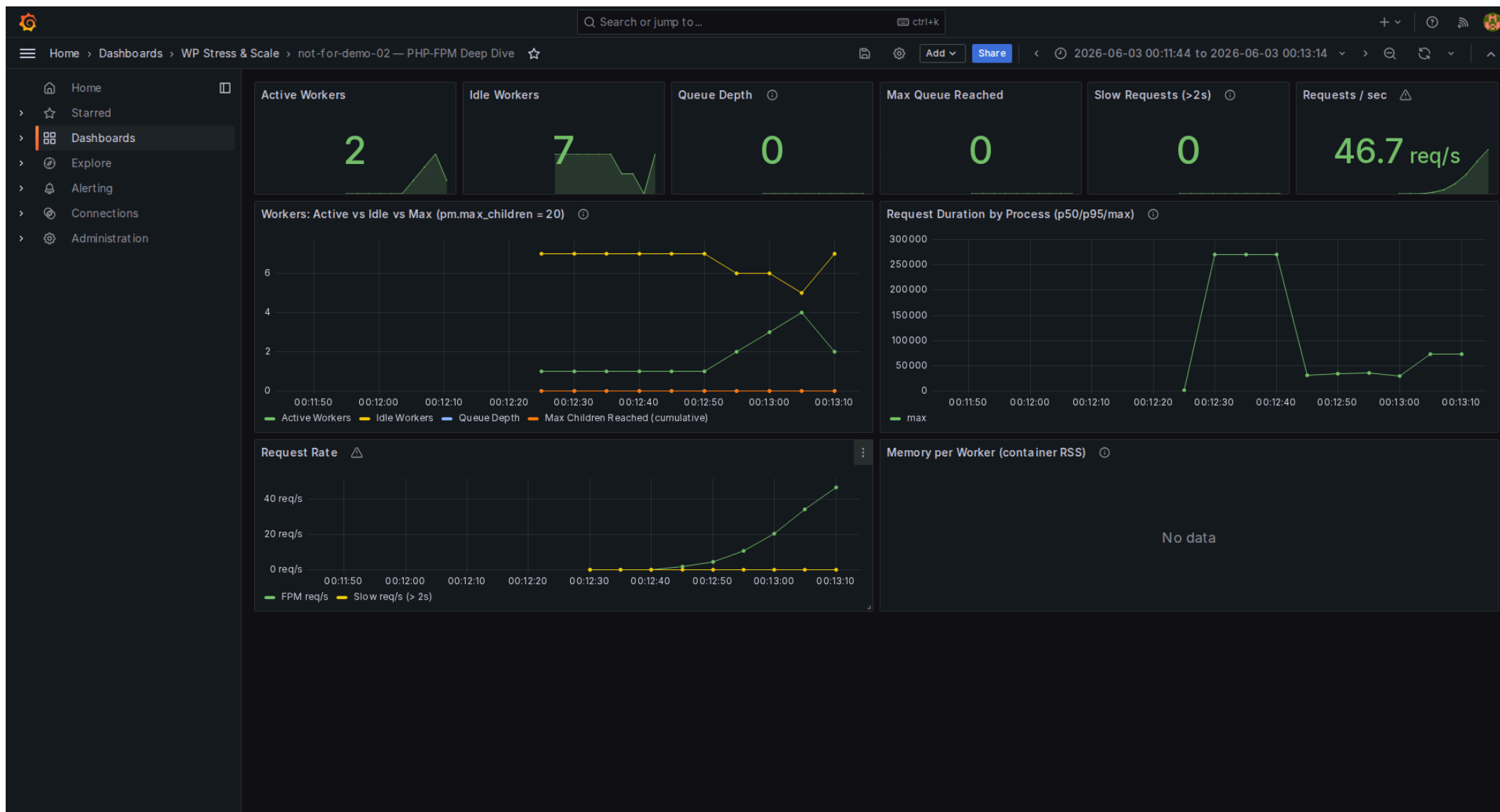
Level 1 — reading the dashboard

Panel	Metric	Value	What it means
Virtual Users	Peak	50	Steady-state load
Requests/s	Peak	~50–55 req/s ★	Stable — no collapse
Latency	p95	~130 ms ★	Was 3+ s — now human-perceptible
FPM Workers	Peak active	~8 / 20	Headroom left
FPM Queue	Depth	0 ★	Crash eliminated — was ~250
MariaDB	Peak threads	~15–20 threads ★	Every request still hitting MySQL
Requests vs			No page cache yet — all to

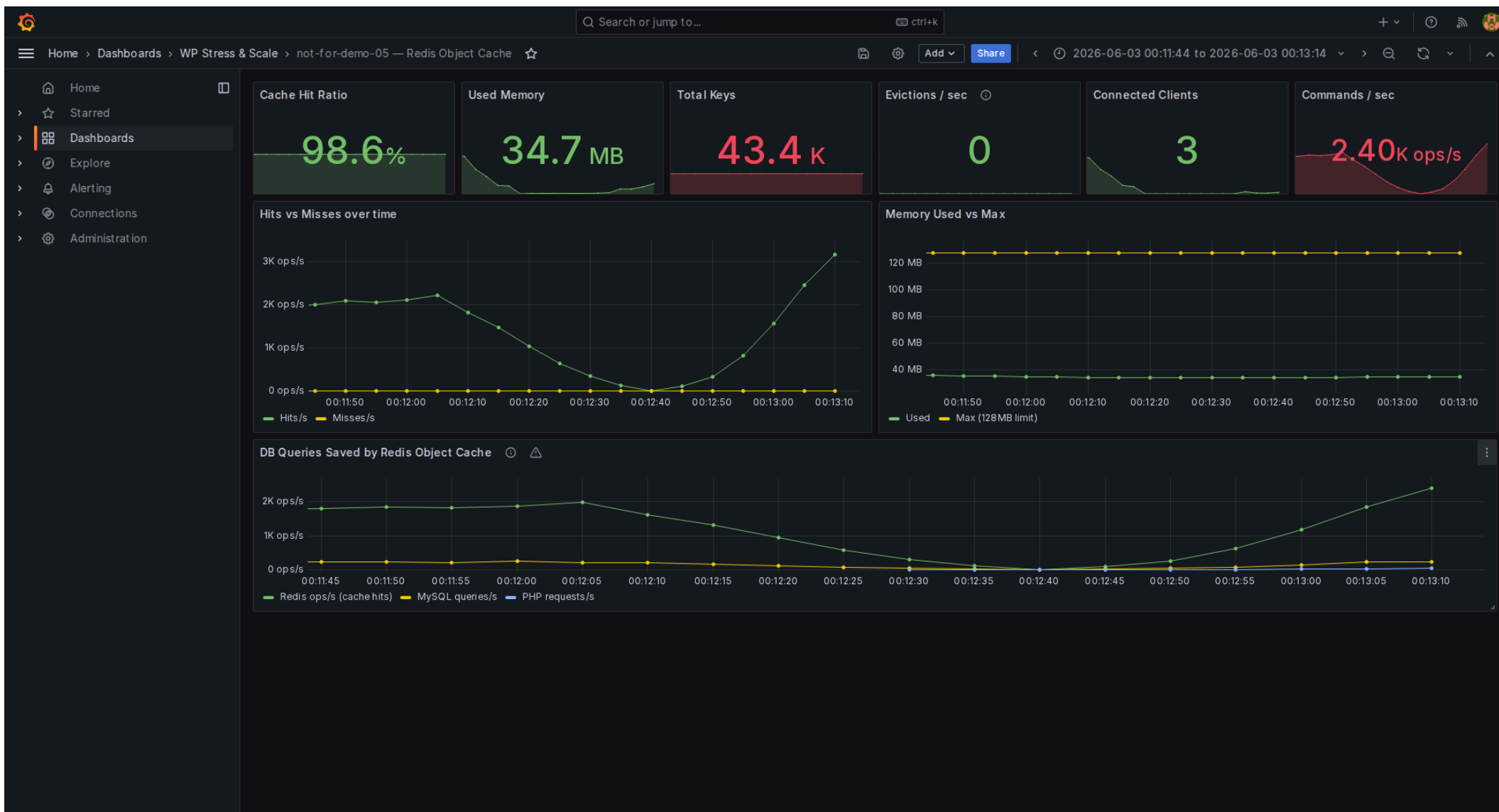
L0 → L1: the jump

Metric	Level 0	Level 1	Δ
Crash at 50 Virtual Users	✗ server buckles	✓ stable	★ crash eliminated
Peak requests/s	~40+ → collapse	~50–55 req/s	stable throughput
p95 Latency	~3 s	~130 ms	★ –96%
p99 Latency	~6 s flat ceiling	~350 ms	–94%
CPU @ 50 Virtual Users	~100%	~25%	★ –75%
Request backlog	~250 (Apache listen queue)	0 (FPM absorbs)	★ queue gone
DB Threads peak	30+	~15–20	still MySQL-heavy

Level 1 — PHP-FPM Workers & Request Rate



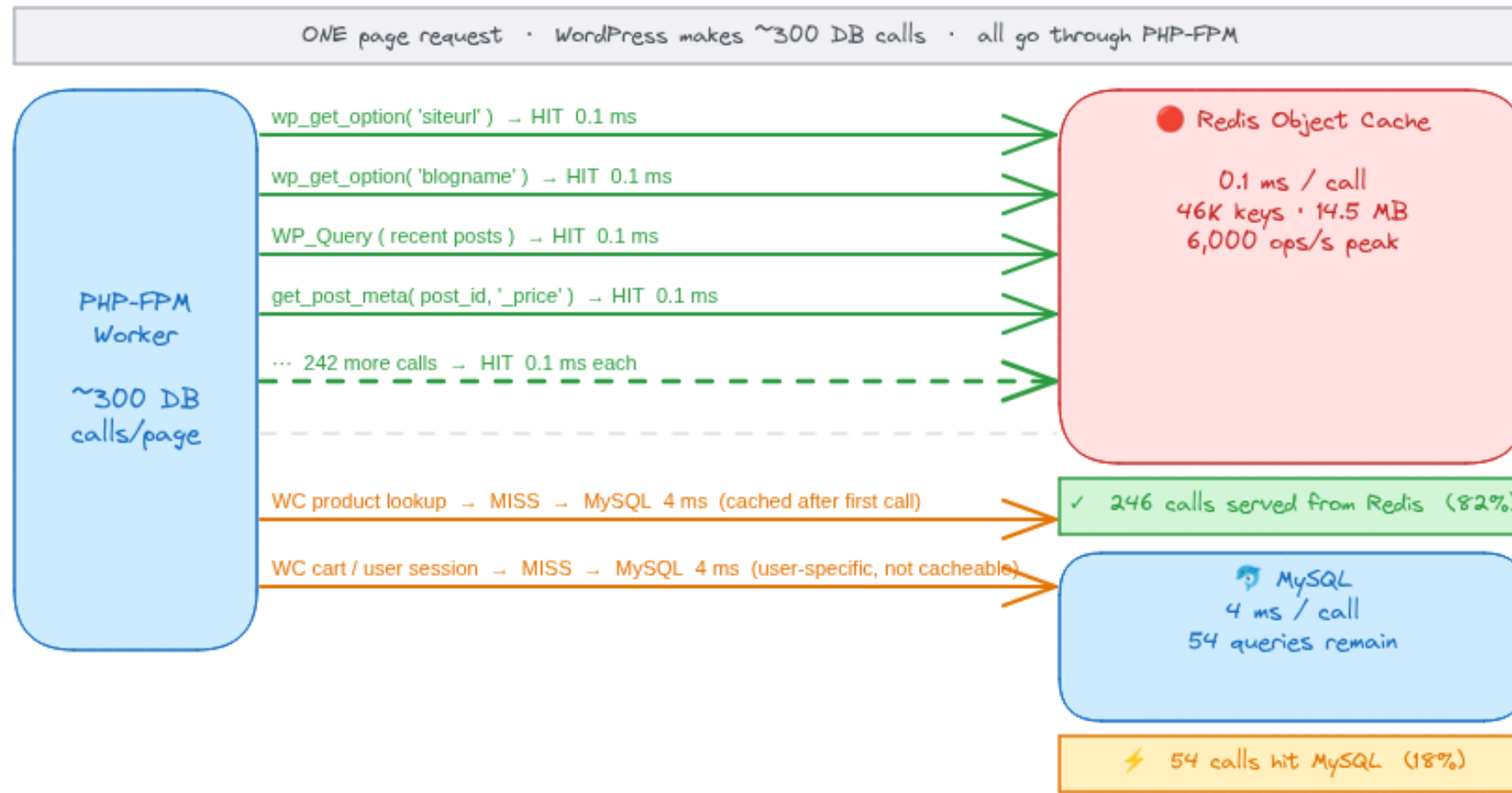
Level 1 — Redis: 97% Hit Rate, ~2.5K ops/s



Level 1 — what Redis is actually doing

~43,000 cached keys · 23 MB · ~2,500 ops/s at peak

Redis Object Cache — What Happens Inside One Page Request





ACT III

Cache Everything

Level 2 — FastCGI page cache + MariaDB tuning

The key insight

Most WordPress page requests return identical HTML for every anonymous visitor.

Home page. Category. Blog post. Shop.

PHP runs. MySQL queries. OPcache helps. Redis helps.

The output is still the same HTML every time.

FastCGI page cache — the logical conclusion:

```
First anonymous visitor → PHP runs → nginx stores response on disk
Every visitor after    → nginx reads cached file → 8ms TTFB
                        PHP never runs
                        MySQL never runs
                        Redis never asked
```

Cached (anonymous visitors) 

Homepage · Category pages · Posts · Shop

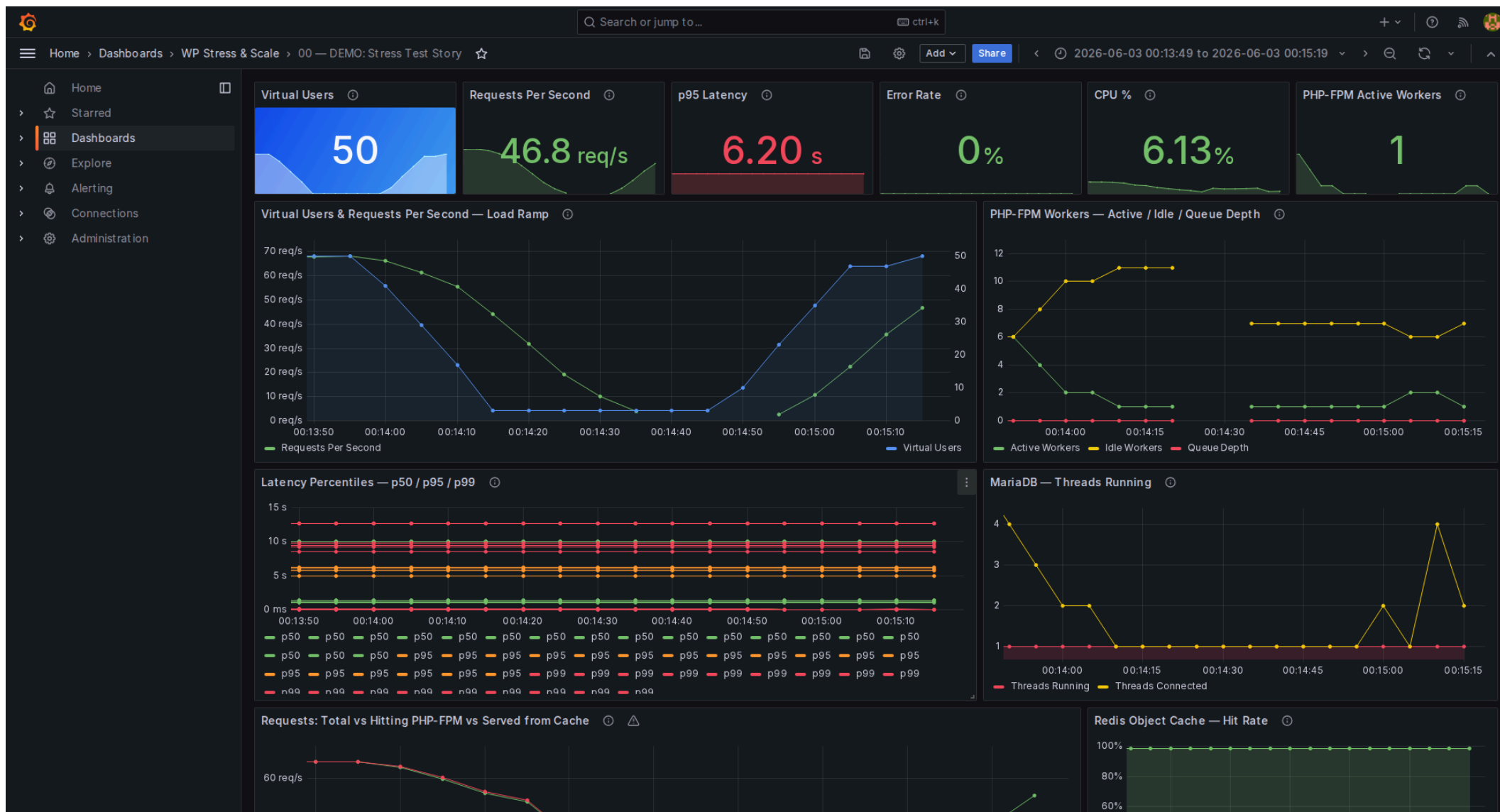
Level 2 — nginx config

```
fastcgi_cache_path /var/cache/nginx
    levels=1:2
    keys_zone=WORDPRESS:100m
    inactive=60m
    max_size=1g;

# Bypass rule — logged-in users and WooCommerce sessions
set $skip_cache 0;
if ($http_cookie ~* "wordpress_logged_in|woocommerce_cart|woocommerce_session") {
    set $skip_cache 1;
}

location ~ /\.php$ {
    fastcgi_cache WORDPRESS;
    fastcgi_cache_key "$scheme$request_method$host$request_uri";
    fastcgi_cache_valid 200 60m;
    fastcgi_cache_bypass $skip_cache;
    fastcgi_no_cache $skip_cache;
    add_header X-Cache-Status $upstream_cache_status; # HIT / MISS / BYPASS
}
```

Level 2 — Demo Dashboard (50 Virtual Users)



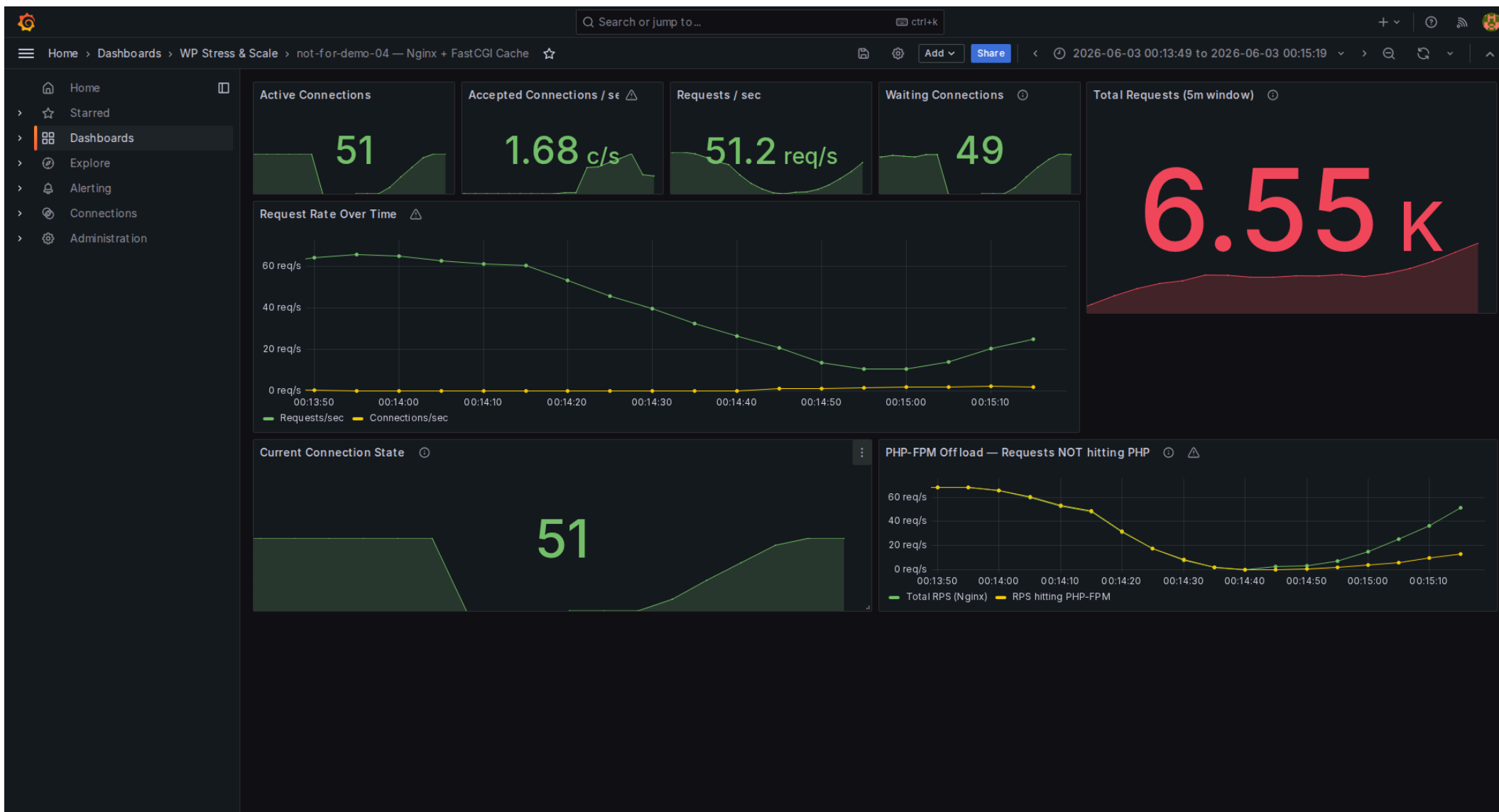
Level 2 — reading the dashboard

Panel	Metric	Value	What it means
Virtual Users	Peak	50	Same load as L1
Requests/s	Peak	~65 req/s ★	~+15–20% vs L1 — nginx serving from disk
Latency	p95	~70–80 ms ★	~-40% vs L1 — most requests never hit PHP
CPU	@ 50 VU	~18%	Was ~25% at L1 — less PHP work
FPM Workers	Peak active	~10–12 / 20	Was ~8 — cache absorbed the burst
FPM Queue	Depth	0 ★	Stays clean — cache absorbs the burst
FPM Request		~20–30%	

L0 → L1 → L2: the progression

Metric	Level 0	Level 1	Level 2
Peak requests/s	~40+ → 🌟	~50–55 req/s	~ 65 req/s
p95 Latency	~3 s 🌟	~130 ms	~ 70–80 ms
CPU @ 50 Virtual Users	~100%	~25%	~ 18%
FPM workers (peak)	—	~8 / 20	~ 10–12 / 20
Request queue	~250 (Apache backlog)	0 (FPM absorbs)	0
DB Threads peak	30+	~15–20	~ 0–3
FastCGI cache offload	0%	0%	~ 70%
Redis hit ratio	—	~80–97%	~ 80–97%

Level 2 — Nginx Cache: FPM Offload in action

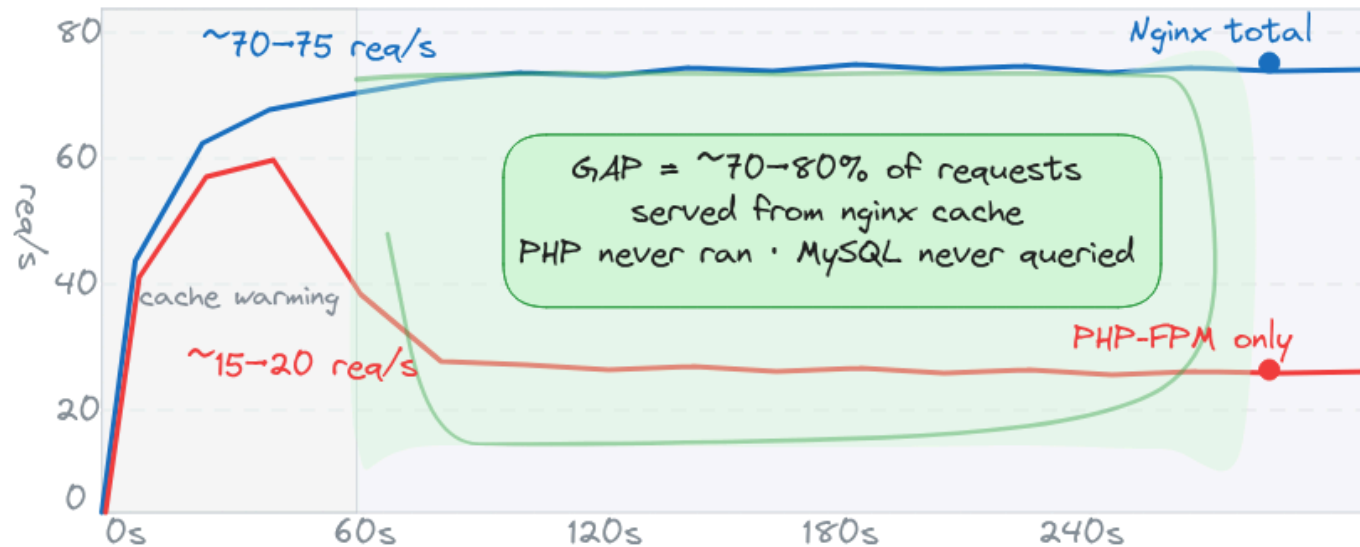


Level 2 — the FPM offload panel explained

~70–75 req/s total · only ~20–30% reach PHP · ~70% gap = cache

Level 2 — FPM Offload Panel: Nginx vs PHP-FPM Request Rate

Grafana panel · 50 Virtual Users · 5-minute test · steady state after ~60s warmup

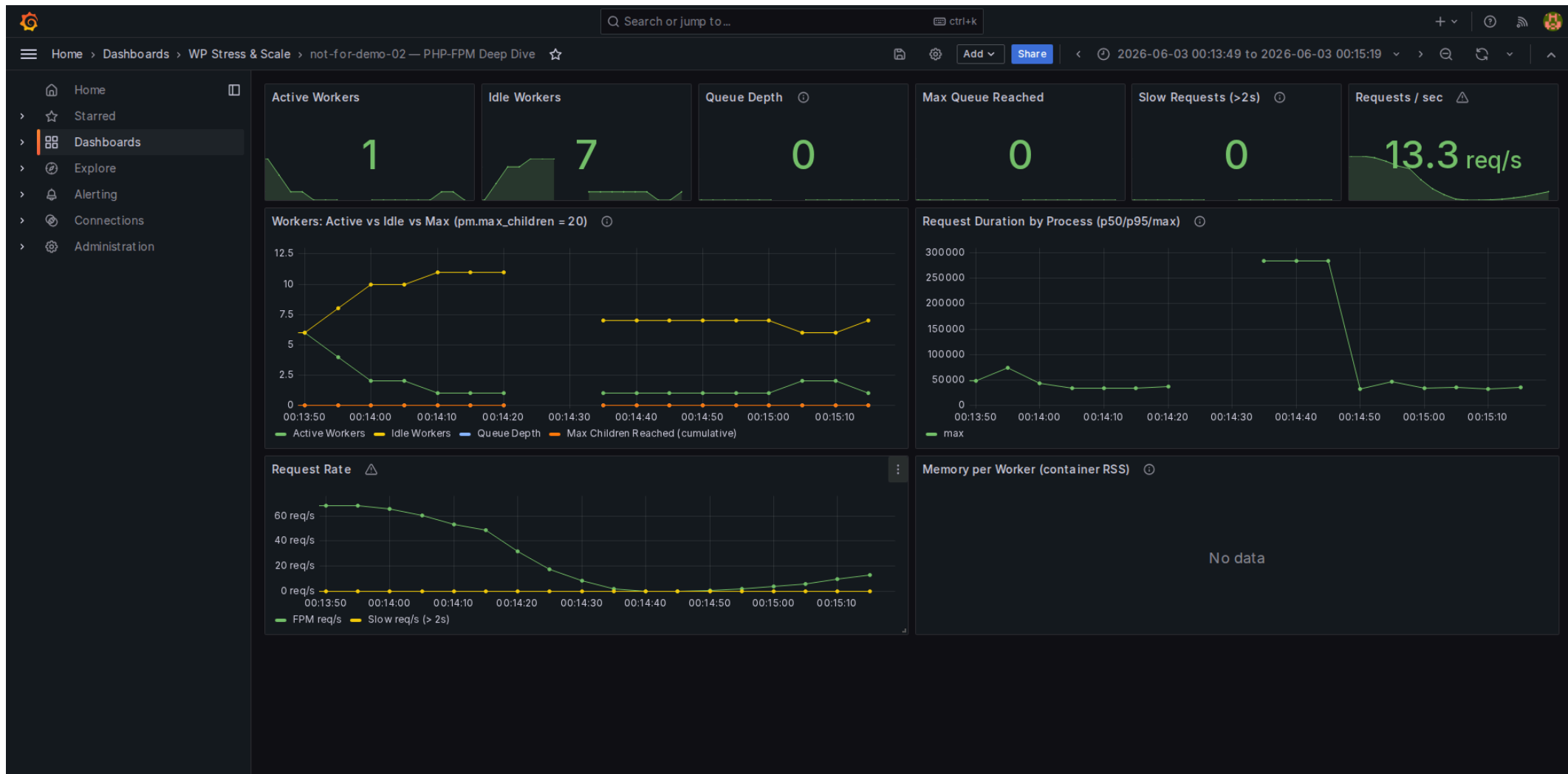


~70 req/s total - ~15-20 req/s to PHP = ~50 req/s answered from disk
2.8x more throughput on same hardware

This is the money slide.

The gap between the two lines is PHP execution that never happened

🔥 Level 2 — PHP-FPM: Peak ~10 Workers During Cache-Miss Burst



MariaDB tuning — Level 2

Level 0 baseline config:

```
# 00-baseline.cnf
innodb_buffer_pool_size = 128M
# (default – fits almost nothing)
max_connections = 151
```

With 128M buffer pool:

- WordPress DB $\approx$ 500MB
- **Every query reads from disk**
- Buffer pool full $\rightarrow$ constant disk I/O
- Slow queries undetected

Level 2 tuned config:

Level 1 vs Level 2 — the real numbers

Metric	Level 1	Level 2	Change
Peak requests/s @ 50 Virtual Users	~50–55 req/s	~65 req/s	~+15–20%
p95 Latency	~130 ms	~70–80 ms	~–40%
CPU usage	~25%	~18%	~–30%
FPM workers (peak)	~8 / 20	~10–12 / 20	cache absorbs the burst
FPM queue depth	0	0	—
MariaDB threads (peak)	~15–20	~0–3	~–85–90%
Cache efficiency	20%	70%	✅ majority served from



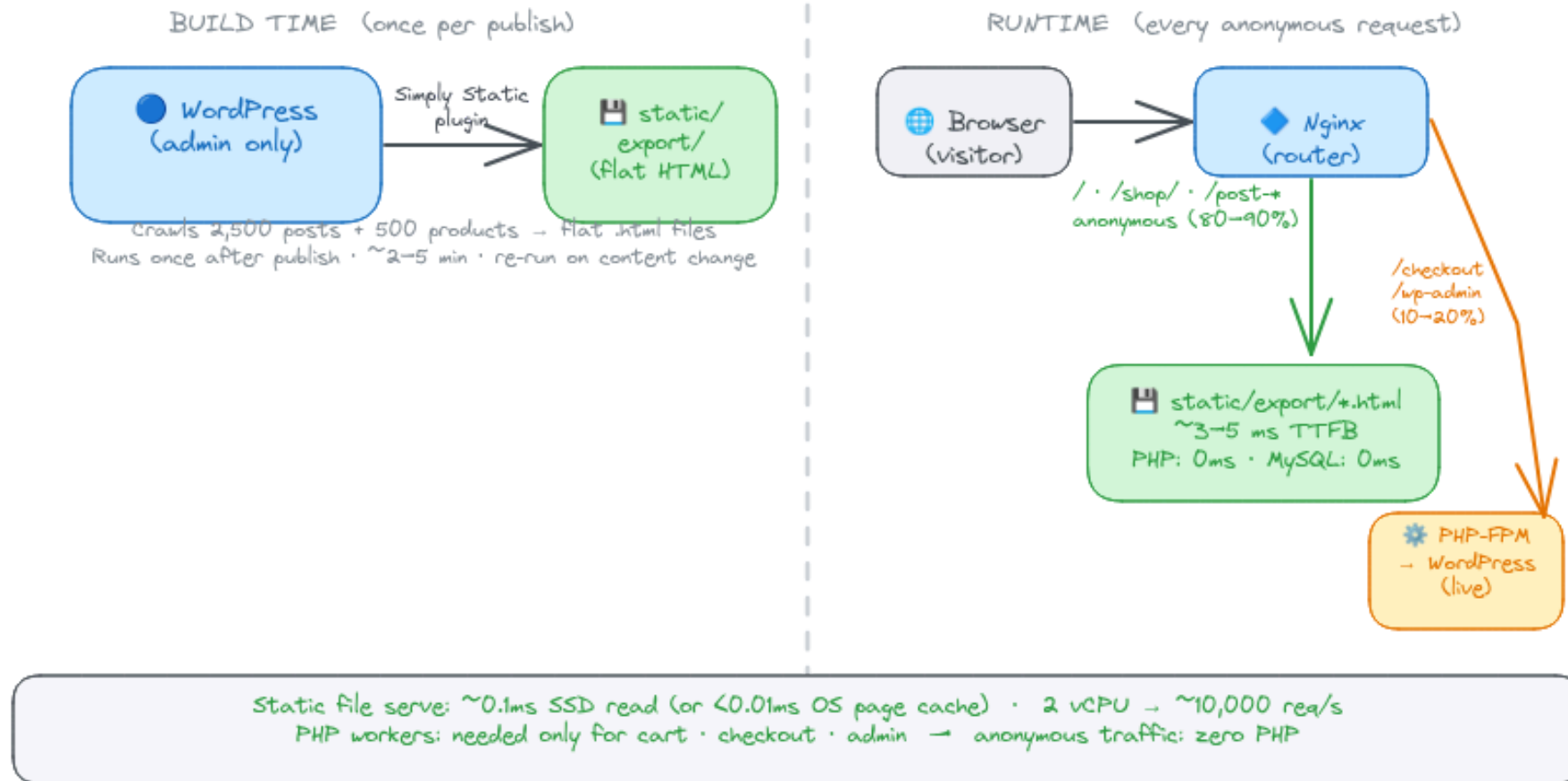
What's Next?

The Static Leap

Level 4 — Simply Static: WordPress as a CMS, not a runtime

Level 4 — the concept

Level 4 — WordPress as a CMS, Nginx as the Runtime



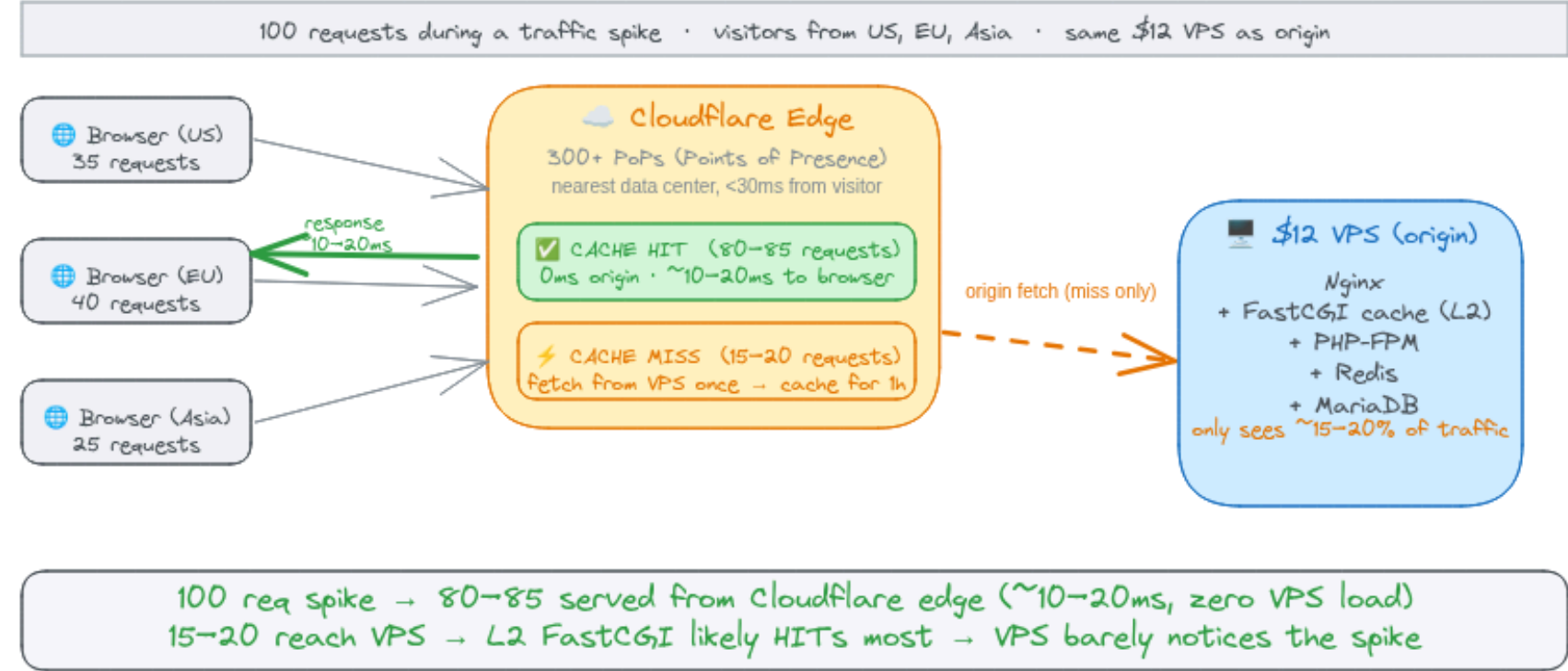
```
wp plugin install simply-static --activate
```

Level 4 — Simply Static: expected numbers

Metric	Level 2	Level 4 (estimated)	Why
requests/s @ 50 Virtual Users	~70–75	500–1,500+	SSD read vs PHP execution
p95 Latency	~70–80 ms	~3–5 ms	file serve vs 300 DB calls
CPU @ 50 Virtual Users	~18%	<3%	no PHP, no MySQL
DB Threads	~3	~0	anonymous traffic never hits DB
PHP workers needed	~11	~0	static files bypass FPM entirely

Level 3 — Cloudflare CDN

Level 3 — Cloudflare CDN: Your VPS Only Sees Cache Misses



Metric	Level 2 (direct)	Level 3 (+ Cloudflare)	Δ
TTFB for cached pages	~70-80 ms	~10-20 ms	~-75-80%

Cloudflare Free Plan — what you get

PoP = Point of Presence — a physical Cloudflare data center. 300+ worldwide means a server is always <30ms from your visitor.

Performance (free)

-  **Global CDN** — 300+ PoPs worldwide
-  **Full page caching** via Cache Rules
-  **Polish** — automatic image compression
-  **Free SSL/TLS** — automatic HTTPS
-  **HTTP/2 + HTTP/3 (QUIC)** — automatic

Security (free)

-  **DDoS mitigation** — unlimited, unmetered
-  **Bot Fight Mode** — basic bot blocking

The full picture

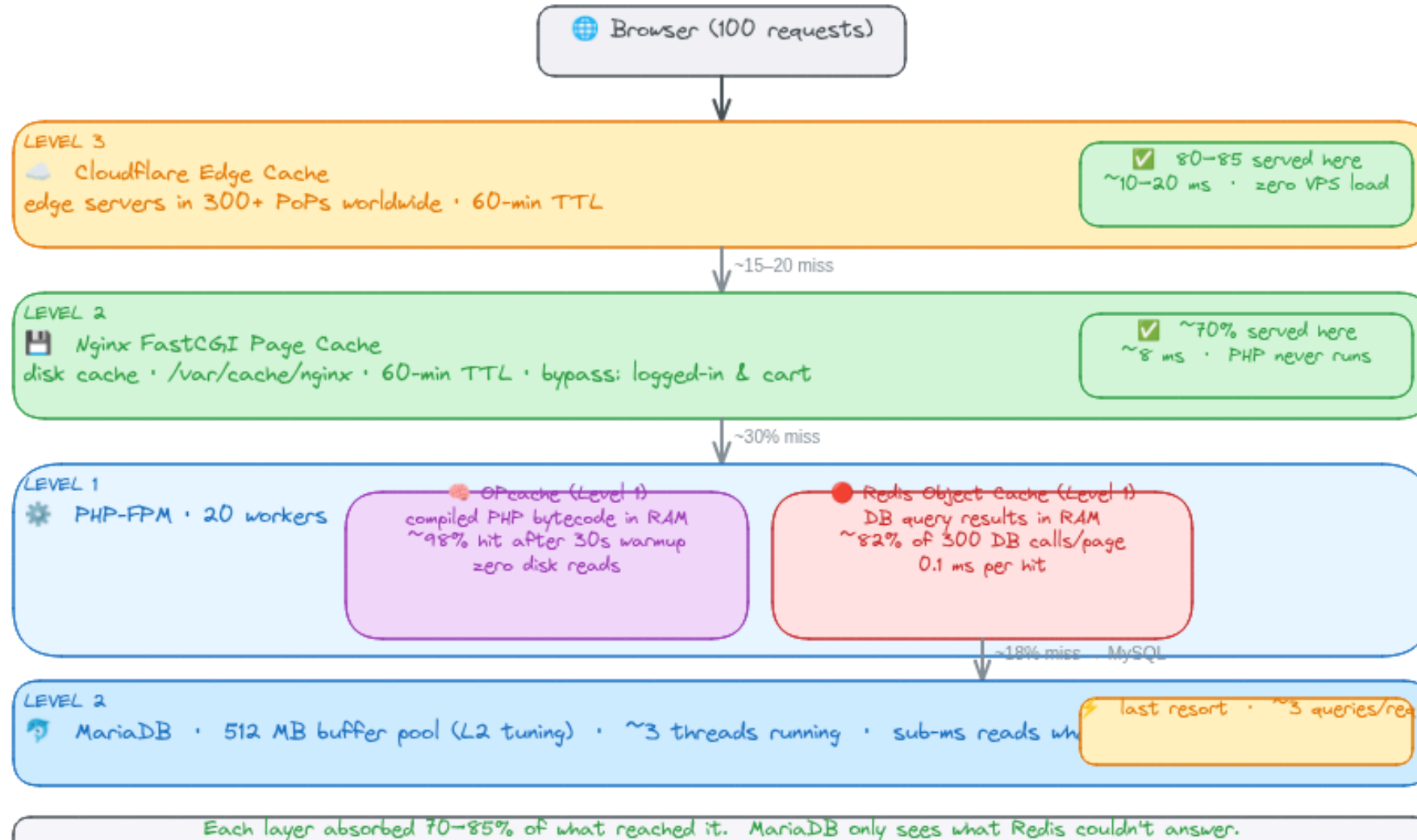
The Full Picture — Same \$12 VPS · \$0 Extra Cost · p95: 3,000 ms → 4 ms



Same \$12 VPS · \$0 extra cost · p95: 3,000 ms → 4 ms = 750x improvement
It was configuration, architecture, and understanding the bottleneck.

The full cache hierarchy

The Full Cache Hierarchy — Every Layer Absorbs What It Can



Lessons learned

1. Measure before you fix

Set up Prometheus + Grafana

before you need it.

You cannot fix what you cannot see.

```
make obs-up — 30 seconds.
```

2. The three WP cache layers

OPcache	→ PHP bytecode	→ always on
Redis	→ DB queries	→ wp redis enable
FastCGI	→ Full pages	→ nginx.conf

Most sites use 0 of these 3.

All three: completely different machine

The tools — all free, all open source

Tool	Role
k6	Load generator — scriptable, Prometheus remote-write
Prometheus	Metrics collection + storage
Grafana	7 dashboards, auto-provisioned
Loki + Promtail	Container log aggregation

Tool	Role
Nginx FastCGI	Full-page cache (built-in)
PHP OPcache	Bytecode cache (built-in)
Redis	Object cache
MariaDB 11	Tuned DB config

Thank you!

Nedko Hristov

Senior DevOps Engineer @ Nemetschek Bulgaria



`github.com/NedkoHristov/WordPress-Europe-2026`

Questions?

*"The best time to add monitoring was before the crash.
The second best time is right now."*